

The background features a large, faint circular logo of Tianjin University on the right, containing the university's name in English and Chinese, and the year 1895. On the left, there is a faint line drawing of a traditional Chinese building with a tiled roof.

内存管理

Memory Management



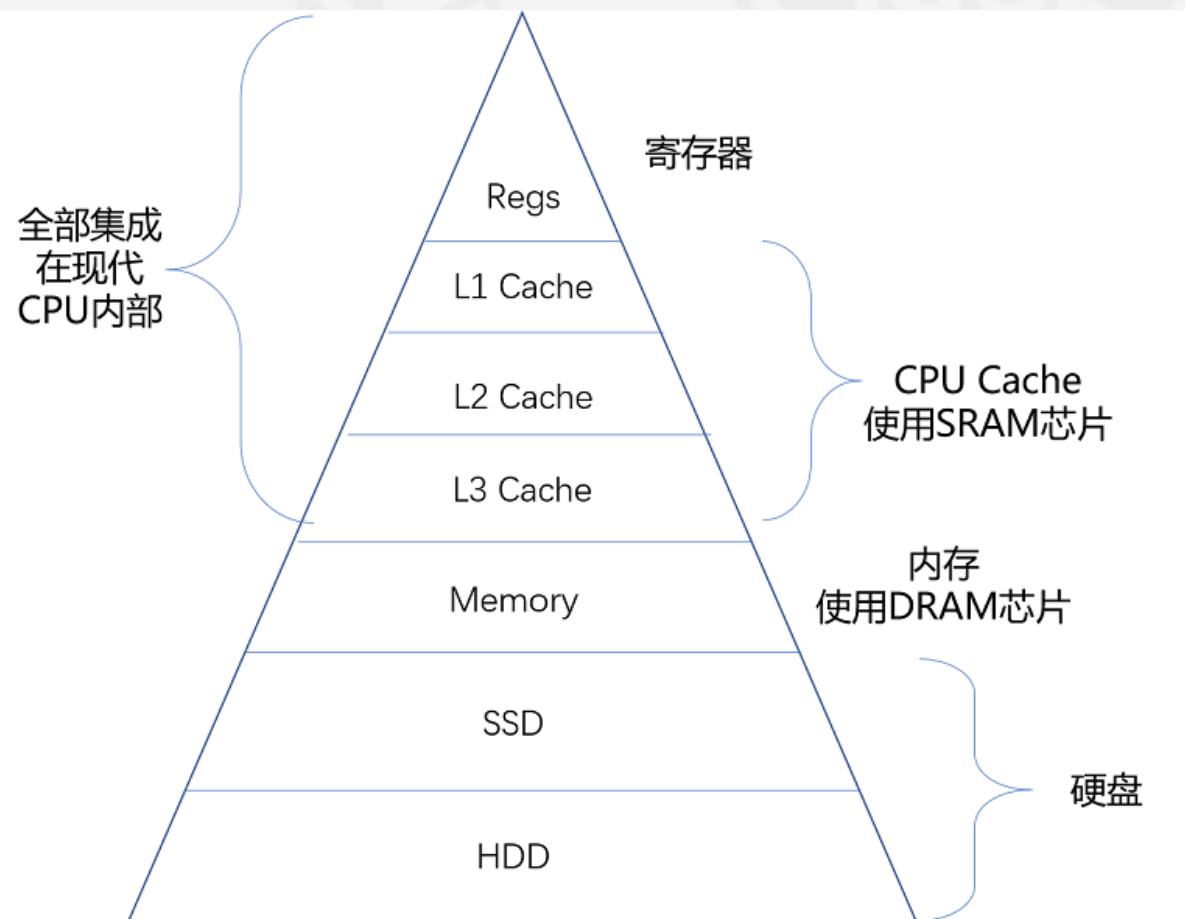
主要内容

- 内存的金字塔模型
- 内存管理单元MMU
- 逻辑与物理地址空间
- 地址重定位

内存金字塔层次 (Memory Hierarchy)

- 从上向下：容量越来越大，速度越来越慢，单位价格越来越低。

存储器	硬件介质	单位成本(美元/MB)	随机访问延时
L1 Cache	SRAM	7	1ns
L2 Cache	SRAM	7	4ns
Memory	DRAM	0.015	100ns
Disk	SSD(NAND)	0.0004	150μs
Disk	HDD	0.00004	10ms





缓存 (Cache)

- 使用**缓存 (Cache)**的目的：在计算机系统中 CPU 的运行速度往往要比内存速度快上好几百倍甚至更多，为了更多地榨取 CPU 的计算能力，就需要在访问数据的速度上进行提升，否则内存的速度将成为整个系统的性能短板。
- 使用**缓存 (Cache)**要考虑的问题
 - 何时将内存中的数据放入缓存。
 - 将数据放入缓存中的什么位置。
 - 当需要更多缓存时，从缓存中移除哪个数据。
 - 被从缓存中移除的数据，应该放在更大的内存中的什么位置。



内存管理单元 The Memory Manager Unit (MMU)

■ 主存 (Primary Memory) 即 RAM

- 保存正在执行的进程所使用的数据和程序
- CPU只能直接处理内存中的数据，不能直接处理辅存中的数据

■ 辅存 (Secondary Memory) 如 硬盘

- 非易失性存储器，用于在进程不执行时存储数据

■ 内存管理单元 (MMU) 目的：管理主存和辅存的使用。

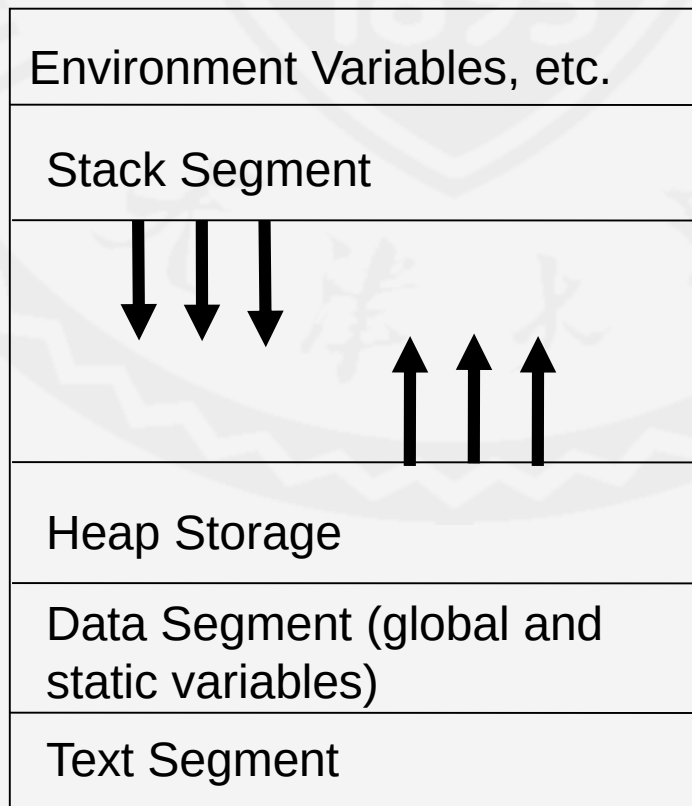
它主要负责：

- 将主存分配给进程
- 在主存和辅存之间移动进程
- 优化内存使用
- 分层存储其实是一种在高速小容量层级的介质层与低速大容量层级的介质层之间进行一种自动或者手动数据迁移、复制、管理等操作的一种存储技术及方案



进程的内存使用模型

- **程序段 (Text)**：程序二进制代码在内存中的映射。
- **数据段 (Data)**：存放全局变量和静态变量，在程序运行初就已分配存储空间，大小固定。
- **栈 (stack)**：存储局部、临时变量，函数调用时存储函数的返回指针，用于控制函数的调用和返回。在程序块开始时自动分配内存，结束时自动释放内存
- **堆 (heap)**：存储动态内存分配，需要程序员手工分配，手工释放。使用`malloc()`和`new`申请到的内存就在堆内。





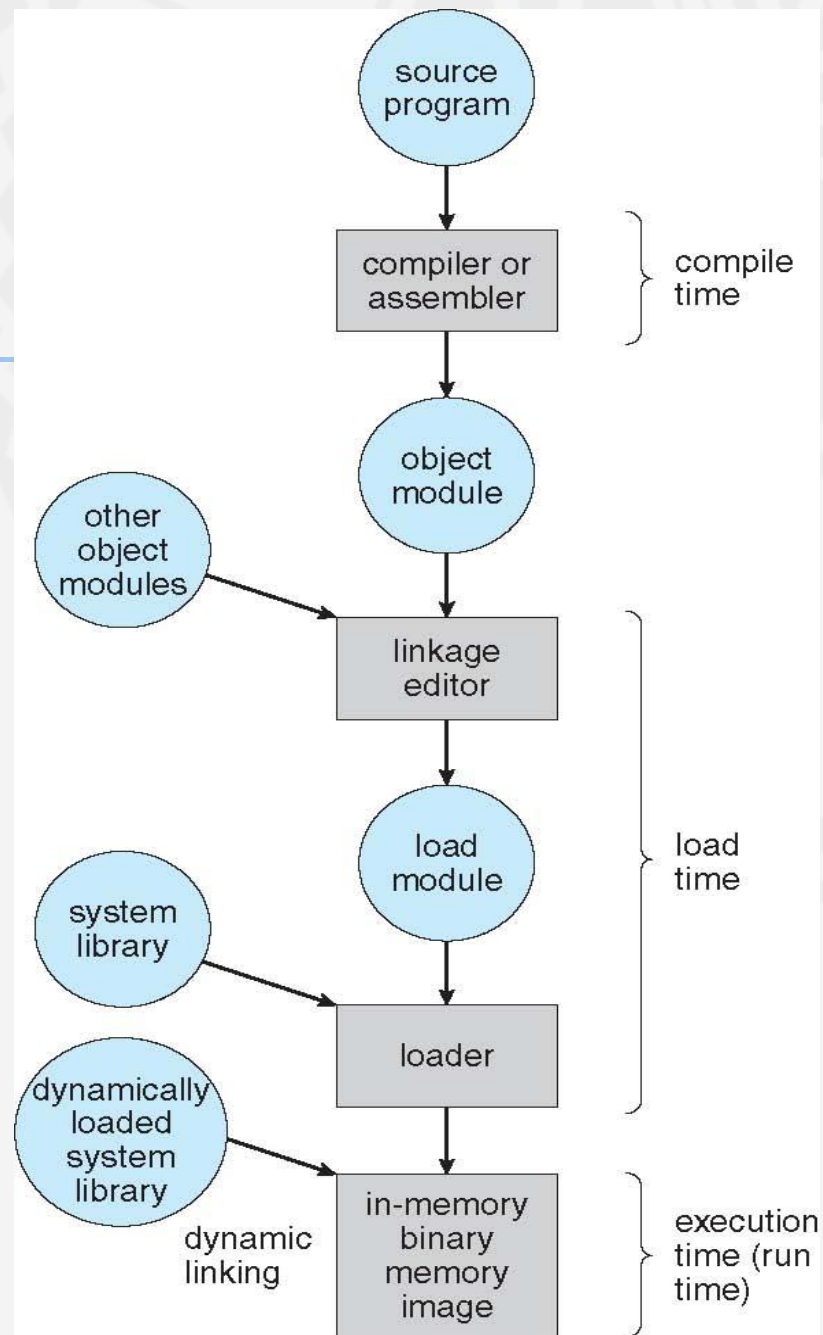
逻辑和物理内存地址空间 (Logical vs. Physical Address Space)

- 逻辑地址空间与独立的物理地址空间绑定的概念是内存管理单元的核心功能
- **逻辑地址 (Logical Address)**：也称为**虚拟地址 (Virtual Address)**。逻辑地址是指计算机中用于访问数据或指令的地址。在操作系统中，逻辑地址是指一个进程所看到的内存地址空间。
- **物理地址 (Physical address)**：物理地址空间是指内存中物理单元的集合。
- 进程只能看到逻辑地址，无法看到物理地址。把逻辑地址转化为实际的物理地址的过程叫做**地址绑定 (Address Binding)**，也称为**地址重定位 (Relocation)**。



执行程序的步骤与地址绑定

- 指令和数据与内存地址的绑定可以在三个不同的阶段发生
 - 编译阶段。如果事先已知内存位置，则可以生成绝对代码；如果起始位置发生变化，则必须重新编译代码。
 - 载入阶段。如果在编译时不知道内存位置，则必须生成可重定位的代码。
 - 执行阶段。如果进程在执行期间可以从一个内存段移动到另一个内存段，则绑定将被延迟到运行时。需要硬件支持地址映射（例如，基址和限值寄存器）





地址重定位 (Relocation)

■ 静态重定位：程序装入内存时。

- 静态重定位是在程序执行之前进行重定位，它根据装配模块将要装入的内存起始位置，直接修改装配模块中的有关使用地址的指令。
- 静态重定位有着无需硬件支持的优点，但存在着如下的缺点：一是程序重定位之后就不能在内存中搬动了；二是要求程序的存储空间是连续的，不能把程序放在若干个不连续的区域。

■ 动态重定位：程序执行时。

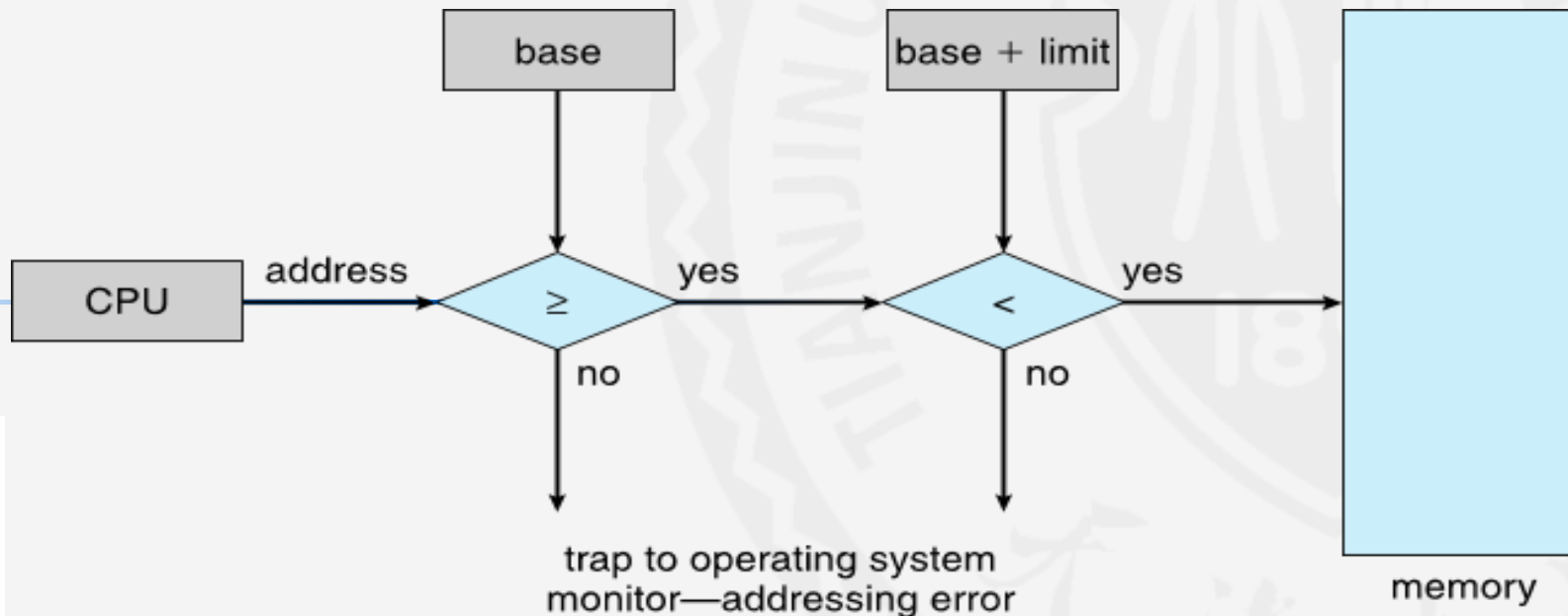
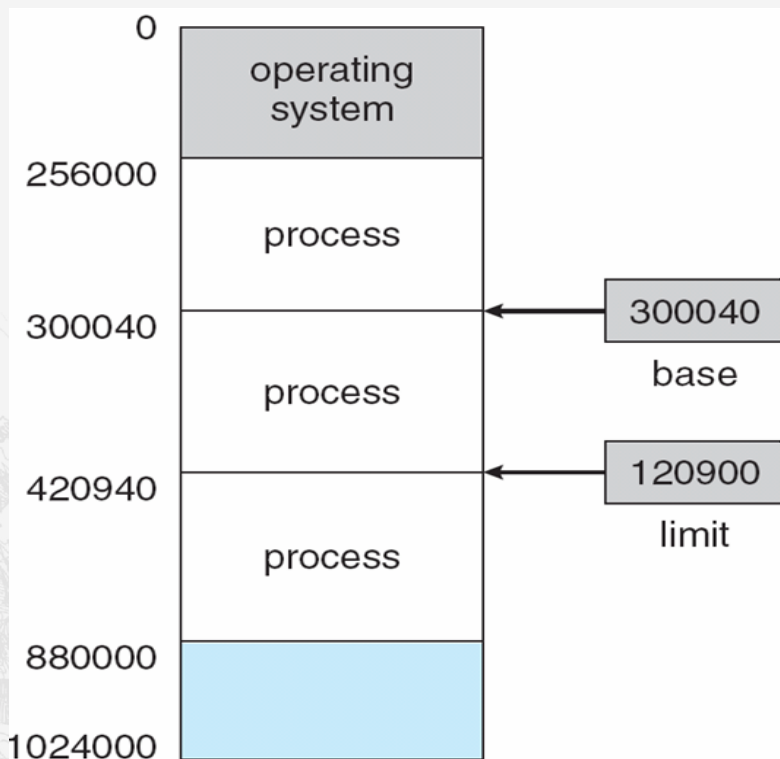
- 动态重定位是指，不是在程序执行之前而是在程序执行过程中进行地址重定位。更确切地说，是在CPU每次访问内存单元前才进行地址变换。动态重定位技术所付出的代价是需要硬件支持。



内存管理

Memory Management

基址和限值寄存器



- 在静态地址重定位中，装入阶段已完成地址转换，执行时使用的地址为物理地址。
- CPU使用使用一对**基址寄存器（Base Register）**和**限值寄存器（Limit Register）**，来检查用户模式下生成的每一个内存访问，以确保其在该用户的基址和限值之间。

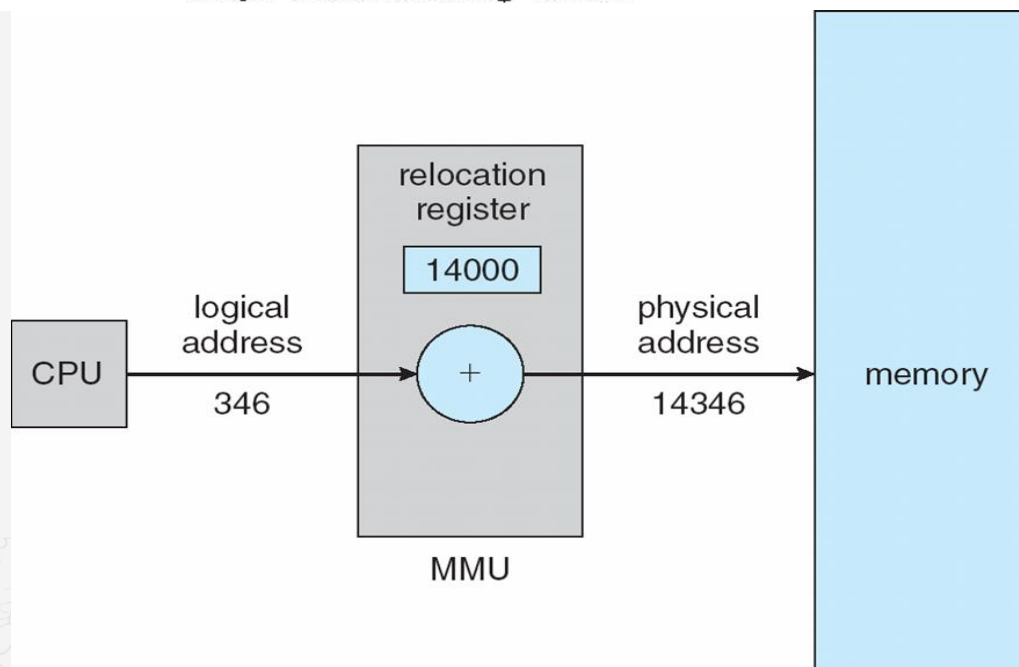
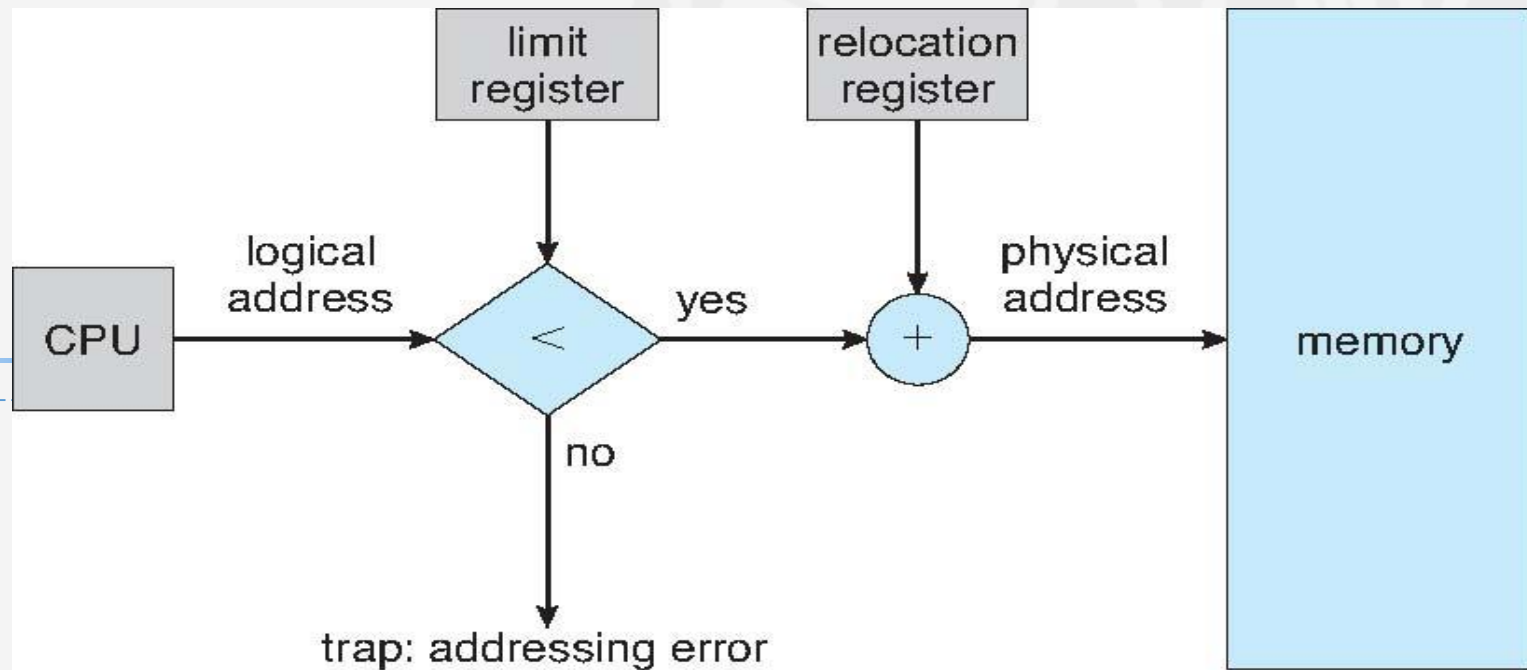


内存管理

Memory Management

重定位和限值寄存器

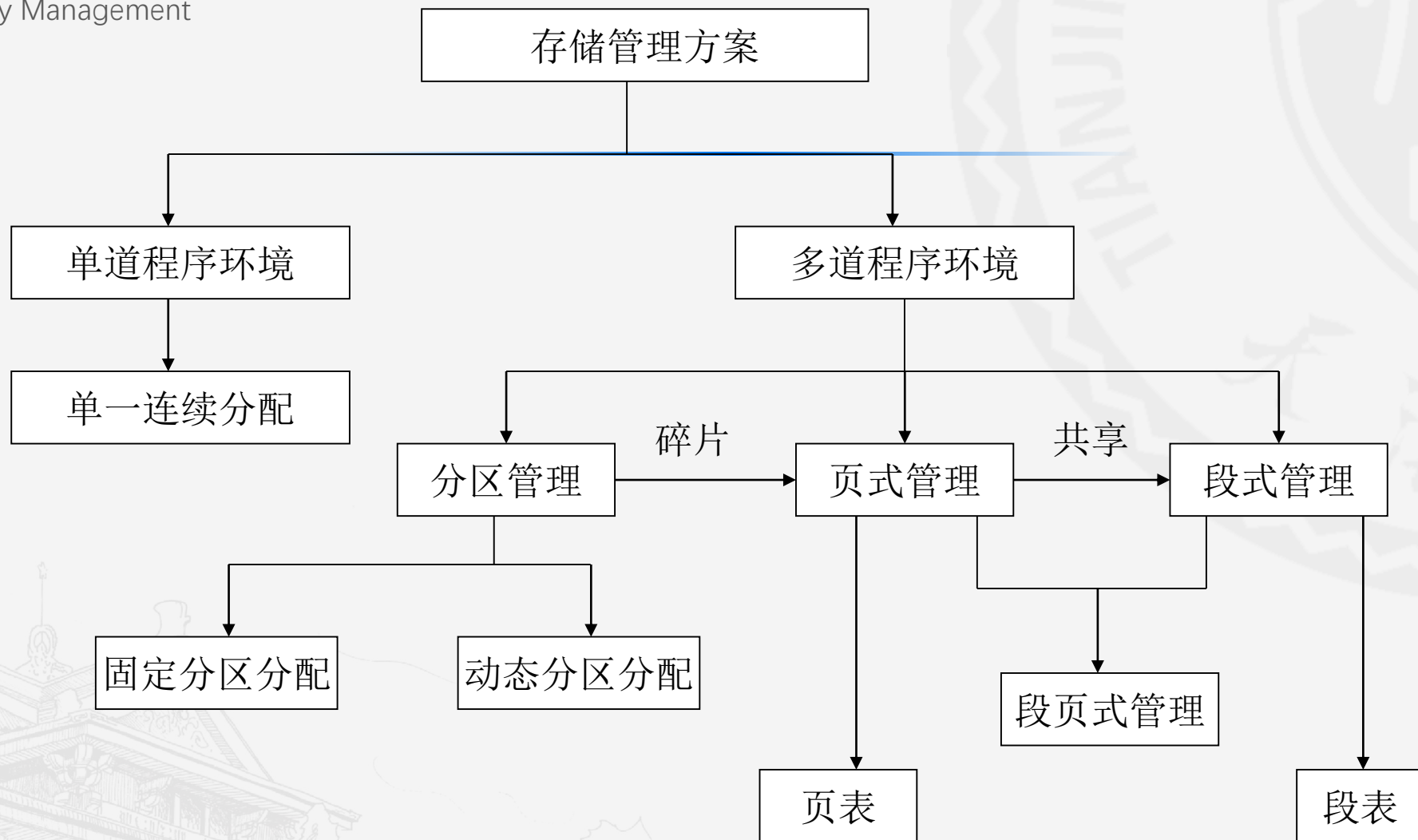
- 动态地址重定位时，地址在执行时才进行转换，CPU检查逻辑地址应小于 **限值寄存器 (Limit Register)**，然后将逻辑地址加上 **重定位寄存器 (Relocation Register)** 得到物理地址。
- 可以提供更好的内存空间利用率：只加载被调用到的例程。也可以提供对动态链接库的支持。





内存管理

Memory Management





研究四类内存管理中要回答的问题

- 如何做地址重定位？
- 如何防止地址越界？
- 如何在有限的物理内存中容纳尽可能多的进程，尽可能大的数据？
- 如何减少对内存的浪费？
- 如何快速高效的管理内存？
- 如何对访问进行权限保护？
- 如何实现共享？



课后阅读

- 《操作系统概念 OSC（第9版）》第8章、第9章。
- 《现代操作系统 MOS（第4版）》第3章。
- 《深入理解计算机系统 CSAPP（第3版）》第6章、第9章。